Contents

Harbor Goods: AWS Well-Architected and DevOps assessment 1
1. Executive summary . . . 1
Top three recommendations . . . 2
2. Scope and method . . . 2
3. Scores . . . 2
Well-Architected Framework pillars . . . 2
DevOps lens . . . 3
4. Findings by pillar . . . 3
4.1 Operational excellence . . . 3
4.2 Security . . . 5
4.3 Reliability . . . 6
4.4 Performance efficiency . . . 8
4.5 Cost optimization . . . 9
4.6 Sustainability . . . 10
4.7 DevOps lens . . . 10
5. Prioritized backlog . . . 13
6. 30/60/90-day roadmap . . . 18
First 30 days (10 items) . . . 18
Days 31 to 60 (26 items) . . . 18
Days 61 to 90 (13 items) . . . 19
7. Evidence register . . . 19
8. Limits and next steps . . . 20

Harbor Goods: AWS Well-Architected and DevOps assessment

> **Fictional sample.** Harbor Goods and all data here are fictional. Each repository in this portfolio is a separate engagement with Harbor Goods, a fictional mid-size retailer. Account IDs are shown as placeholders. Every finding and figure in this report comes from the synthetic data in `data/synthetic/`.

Item	Detail
Client	Harbor Goods (fictional mid-size retailer)
Workload	`storefront-orders`: storefront, order API and fulfilment worker
Accounts	Production `<account-id>`, staging `<account-id>`, shared tooling `<account-id>` (placeholders)
Region	`us-east-1`
Lenses	AWS Well-Architected Framework (six pillars) and the DevOps lens (AWS Well-Architected DevOps Guidance)
Access used	Read-only for infrastructure inspection; Tool record kept by the client
Revision	1.0

Tables between `BEGIN GENERATED` and `END GENERATED` markers in the source come from `make evidence` and the data. The tests in `tests/test_report.py` recompute them.

1. Executive summary

Harbor Goods ships changes often, but a failed change is expensive: releases go out to every user at once, nothing checks them after they land, and recovery waits for a customer to complain. The same pattern shows up in the

data layer. The orders database has backups that nobody has restored and a single writer, and no one has agreed how long checkout may be down.

Most fixes are configuration and process changes on the existing architecture. The team already keeps everything in version control, runs production in its own account, encrypts data at rest, and builds on managed services.

Measure	Value
Well-Architected score (six pillars)	39%
DevOps lens score	38%
High-risk issues (HRI), framework and DevOps lens	7 and 1
Medium-risk issues (MRI), framework and DevOps lens	13 and 8
Best practices reviewed	58
Backlog items (high, medium, low)	49 (9, 27, 13)
Items planned for 30, 60 and 90 days	10, 26, 13
Deployments per week	3.7
Change failure rate	10.6%
Median time to restore service	70 minutes

Top three recommendations

1. **Plan for unsuccessful changes** (OPS06-BP01, High risk, effort S). Add a rollback section to the pull request template, require backward-compatible (expand and contract) schema changes, and keep the previous task definition revision ready to redeploy.
2. **Use temporary credentials** (SEC02-BP02, High risk, effort S). Replace the ci-deployer key with GitHub OIDC federation to an IAM role scoped to the repository and the production environment, then deactivate and delete the key.
3. **Perform periodic recovery of the data to verify backup integrity and processes** (REL09-BP04, High risk, effort S). Restore the latest snapshot into the staging account every quarter, time it, check row counts, and record the result next to the recovery runbook.

2. Scope and method

The review covered one workload, `storefront-orders`, across the production, staging and shared tooling accounts. Store point-of-sale systems, the corporate data warehouse and penetration testing were out of scope.

The assessment followed the method in `docs/methodology.md`:

1. A scoping workshop agreed the workload, the people to interview and read-only access.
2. A two-hour interview walked through each best practice in scope.
3. Read-only reviews and exports checked each answer against what is running (section 7 lists the evidence).
4. The assessor recorded every answer against its best practice with a status, a risk rating, an effort estimate and an owner.
5. Scripts scored the answers, built the backlog and scheduled the roadmap from fixed rules.

Scores work as follows. A best practice earns 1 when met, 0.5 when partial and 0 when not met; the pillar score is the share earned. A question is a high-risk issue (HRI) when the assessor rates any gap under it high, and a medium-risk issue (MRI) when its worst gap is medium, which mirrors how the Well-Architected Tool reports risk per question. Maturity levels run from 1 (Initial, below 40%) to 4 (Optimized, 80% and above).

3. Scores

Well-Architected Framework pillars

Pillar	Score	Maturity	Met	Partial	Not met	HRI	MRI
Operational excellence	46%	2 Repeatable	2	7	3	1	5
Security	39%	1 Initial	2	3	4	2	4

Pillar	Score	Maturity	Met	Partial	Not met	HRI	MRI
Reliability	31%	1 Initial	1	3	4	4	1
Performance efficiency	38%	1 Initial	1	1	2	0	1
Cost optimization	30%	1 Initial	0	3	2	0	2
Sustainability	50%	2 Repeatable	1	1	1	0	0

DevOps lens

Lens	Score	Maturity	Met	Partial	Not met	HRI	MRI
DevOps lens	38%	1 Initial	2	9	6	1	8

Reliability and security carry most of the high risks. Cost optimization scores low but holds no high risk; the gaps there are tasks larger than their load, idle resources and On-Demand pricing.

4. Findings by pillar

Each finding names the best practice, its status, the risk if it stays open, the evidence behind it and the change that closes it. Backlog keys (`HG-nn`) link findings to section 5.

4.1 Operational excellence

Question	Text	Risk
OPS04	How do you implement observability in your workload?	Medium
OPS05	How do you reduce defects, ease remediation, and improve flow into production?	Medium
OPS06	How do you mitigate deployment risks?	High
OPS07	How do you know that you are ready to support a workload?	Medium
OPS08	How do you utilize workload observability in your organization?	None
OPS10	How do you manage workload and operations events?	Medium
OPS11	How do you evolve operations?	Medium

OPS04-BP01 Identify key performance indicators Status: Partial. Risk: Medium. Effort: S. Owner: Head of engineering. Backlog: HG-10. Evidence: EV-01, EV-09.

The team tracks conversion rate in the marketing tool but no operational KPI such as checkout success rate or order latency. Dashboards show CPU and memory only.

Recommendation: Agree three KPIs with the business (checkout success rate, p95 checkout latency, orders sent to the logistics partner within five minutes) and publish them on one CloudWatch dashboard.

OPS05-BP01 Use version control Status: Met. Evidence: EV-04, EV-08.

Application code, Terraform and pipeline definitions all live in GitHub with branch protection.

OPS05-BP02 Test and validate changes Status: Partial. Risk: Medium. Effort: M. Owner: Backend engineers. Backlog: HG-24. Evidence: EV-04.

Unit tests run on pull requests for orders-api only. storefront-web and the fulfilment worker merge without automated tests, and nothing tests database migrations before production.

Recommendation: Run unit tests for all three services on every pull request, and add a migration test that applies each schema change to a copy of the staging database.

OPS05-BP04 Use build and deployment management systems Status: Met. Evidence: EV-04.

GitHub Actions builds and deploys every service; nobody builds images on a laptop.

OPS05-BP10 Fully automate integration and deployment Status: Partial. Risk: Medium. Effort: M. Owner: Platform lead. Backlog: HG-25. Evidence: EV-04, EV-06.

Deployments to production need an engineer to run a workflow by hand and then run the database migration from a laptop. The median lead time from merge to production is 67.9 hours.

Recommendation: Promote the same image from staging to production in one pipeline, run migrations as a pipeline step, and keep the manual approval as the only human action.

OPS06-BP01 Plan for unsuccessful changes Status: Not met. Risk: High. Effort: S. Owner: Platform lead. Backlog: HG-01. Evidence: EV-02, EV-06, EV-07.

No change has a written rollback plan. Engineers rolled back 3 of 47 deployments in the window by hand, and the longest incident, INC-106, lasted 185 minutes because nobody could reverse its schema change.

Recommendation: Add a rollback section to the pull request template, require backward-compatible (expand and contract) schema changes, and keep the previous task definition revision ready to redeploy.

OPS06-BP03 Employ safe deployment strategies Status: Not met. Risk: Medium. Effort: M. Owner: Platform lead. Backlog: HG-26. Evidence: EV-04, EV-13.

ECS rolling updates replace all tasks at once with minimumHealthyPercent set to 0.

Recommendation: Use ECS blue/green deployments through CodeDeploy, or at least rolling updates with minimumHealthyPercent 100 and the deployment circuit breaker enabled.

OPS06-BP04 Automate testing and rollback Status: Not met. Risk: High. Effort: M. Owner: Platform lead. Backlog: HG-05. Evidence: EV-04, EV-06.

Nothing checks a release after it ships. The 3 rollbacks in the window all started after a customer or a staff member noticed the problem.

Recommendation: Enable the ECS deployment circuit breaker with rollback, and add a post-deploy smoke test and a CloudWatch alarm on the checkout KPI that stops and reverts the deployment.

OPS07-BP03 Use runbooks to perform procedures Status: Partial. Risk: Medium. Effort: S. Owner: Platform lead. Backlog: HG-11. Evidence: EV-02.

Two runbooks exist in a wiki (restart a service, rotate the logistics API token), and nobody used either during INC-105. Database failover and restore have no runbook.

Recommendation: Write runbooks for the five most frequent procedures, keep them next to the code, and link each alarm to its runbook.

OPS08-BP01 Analyze workload metrics Status: Partial. Risk: Low. Effort: S. Owner: Head of engineering. Backlog: HG-37. Evidence: EV-09.

Metrics exist but nobody reviews them outside incidents.

Recommendation: Hold a 30-minute weekly operations review of the KPI dashboard and the alarm history.

OPS10-BP01 Use a process for event, incident, and problem management Status: Partial. Risk: Medium. Effort: S. Owner: Head of engineering. Backlog: HG-12. Evidence: EV-03, EV-07.

The team handles incidents in a chat channel with no severity definitions or incident lead. 3 of 7 incidents in the window were first reported by customers.

Recommendation: Define three severity levels, an incident lead role and a status page update rule, and page the on-call engineer from CloudWatch alarms.

OPS11-BP02 Perform post-incident analysis Status: Partial. Risk: Medium. Effort: S. Owner: Head of engineering. Backlog: HG-13. Evidence: EV-07.

Only 2 of 7 incidents have a written post-incident analysis, and their actions are not tracked.

Recommendation: Write a blameless analysis for every SEV1 and SEV2 incident within five working days and track its actions in the team backlog.

4.2 Security

Question	Text	Risk
SEC01	How do you securely operate your workload?	None
SEC02	How do you manage authentication for people and machines?	High
SEC03	How do you manage permissions for people and machines?	High
SEC04	How do you detect and investigate security events?	Medium
SEC06	How do you protect your compute resources?	Medium
SEC08	How do you protect your data at rest?	None
SEC10	How do you anticipate, respond to, and recover from incidents?	Medium
SEC11	How do you incorporate and validate the security properties of applications throughout the design, development, and deployment lifecycle?	Medium

SEC01-BP01 Separate workloads using accounts Status: Met. Evidence: EV-01, EV-08.

Production, staging and shared tooling run in separate accounts in one AWS Organizations organization.

SEC02-BP02 Use temporary credentials Status: Not met. Risk: High. Effort: S. Owner: Platform lead. Backlog: HG-02. Evidence: EV-04, EV-05.

GitHub Actions deploys with the long-lived access key of the IAM user ci-deployer, stored as a repository secret. 3 active access keys in production are older than 90 days; the oldest is 806 days old.

Recommendation: Replace the ci-deployer key with GitHub OIDC federation to an IAM role scoped to the repository and the production environment, then deactivate and delete the key.

SEC02-BP03 Store and use secrets securely Status: Partial. Risk: Medium. Effort: S. Owner: Backend engineers. Backlog: HG-14. Evidence: EV-02, EV-13.

The database password sits in AWS Secrets Manager, but the logistics partner API token is a plain-text ECS environment variable, and the team rotated it by hand after it expired (INC-105).

Recommendation: Move the partner token to Secrets Manager, inject it through the task definition `secrets` field, and set a rotation reminder or automatic rotation.

SEC03-BP02 Grant only the access each role needs Status: Not met. Risk: High. Effort: M. Owner: Platform lead. Backlog: HG-06. Evidence: EV-05, EV-08.

ci-deployer and ops-admin-2 hold AdministratorAccess. 1 console user in production has no MFA.

Recommendation: Give the new deploy role only the ECS, ECR and Terraform state permissions it uses, move people to IAM Identity Center permission sets, and enforce MFA.

SEC04-BP01 Configure service and application logging Status: Partial. Risk: Medium. Effort: S. Owner: Platform lead. Backlog: HG-15. Evidence: EV-09, EV-12.

An organization CloudTrail trail exists, but ALB access logs are off and the Aurora PostgreSQL log is not exported to CloudWatch Logs.

Recommendation: Enable ALB access logs to S3 and export the Aurora PostgreSQL log, each with a retention period that matches the investigation needs.

SEC06-BP01 Perform vulnerability management Status: Partial. Risk: Medium. Effort: S. Owner: Platform lead. Backlog: HG-16. Evidence: EV-12.

Amazon Inspector scans ECR images, but its findings go to nobody and the storefront image carries 11 critical findings with fixes available.

Recommendation: Route Inspector critical and high findings to the team channel, rebuild images weekly on a patched base image, and fail the pipeline on critical findings that have a fix.

SEC08-BP02 Enforce encryption at rest Status: Met. Evidence: EV-10, EV-12.

AWS KMS keys encrypt Aurora, ElastiCache, SQS and S3; default EBS encryption is on.

SEC10-BP02 Develop incident management plans Status: Not met. Risk: Medium. Effort: S. Owner: Head of engineering. Backlog: HG-17. Evidence: EV-02.

The team has no security incident plan and did not know who would decide to rotate every credential.

Recommendation: Write a one-page security incident plan (roles, contacts, first actions for a leaked key) and rehearse it once as a tabletop exercise.

SEC11-BP02 Automate testing throughout the development and release lifecycle Status: Not met. Risk: Medium. Effort: M. Owner: Backend engineers. Backlog: HG-27. Evidence: EV-04.

No static analysis, dependency scanning or infrastructure-as-code scanning runs in the pipelines.

Recommendation: Add Semgrep, dependency scanning and Checkov to pull request checks, starting in report-only mode and failing on new high findings after two weeks.

4.3 Reliability

Question	Text	Risk
REL06	How do you monitor workload resources?	High

Question	Text	Risk
REL07	How do you design your workload to adapt to changes in demand?	Medium
REL09	How do you back up data?	High
REL10	How do you use fault isolation to protect your workload?	High
REL12	How do you test reliability?	None
REL13	How do you plan for disaster recovery (DR)?	High

REL06-BP01 Monitor all components for the workload (Generation) Status: Partial. Risk: High. Effort: M. Owner: Platform lead. Backlog: HG-07. Evidence: EV-03, EV-07, EV-09.

Alarms cover ALB 5xx errors only. The SQS dead-letter queue, Aurora connections, Lambda errors and the logistics partner API have no alarm, and customers found 3 of 7 incidents first, with a median time to detect of 30 minutes.

Recommendation: Add alarms for the dead-letter queue depth, Aurora CPU and connections, Lambda errors and throttles, and a CloudWatch Synthetics canary on the checkout path.

REL07-BP01 Use automation when obtaining or scaling resources Status: Partial. Risk: Medium. Effort: S. Owner: Platform lead. Backlog: HG-18. Evidence: EV-13, EV-07.

storefront-web scales on CPU; orders-api runs a fixed two tasks and ran out of memory under load (INC-101).

Recommendation: Add target tracking scaling to orders-api on CPU and on ALB requests per target, with a minimum of three tasks across three Availability Zones.

REL09-BP01 Identify and back up all data that needs to be backed up, or reproduce the data from sources Status: Met. Evidence: EV-10.

Aurora automated backups (seven days) and an AWS Backup plan cover the orders database and the image bucket.

REL09-BP04 Perform periodic recovery of the data to verify backup integrity and processes Status: Not met. Risk: High. Effort: S. Owner: Platform lead. Backlog: HG-03. Evidence: EV-10.

Nobody has ever restored the orders database from a backup, and the restore time is unknown.

Recommendation: Restore the latest snapshot into the staging account every quarter, time it, check row counts, and record the result next to the recovery runbook.

REL10-BP01 Deploy the workload to multiple locations Status: Partial. Risk: High. Effort: M. Owner: Platform lead. Backlog: HG-08. Evidence: EV-10, EV-13.

ECS tasks span two Availability Zones, but the Aurora cluster has a single writer and no reader, so a writer failure means recreating the instance, and ElastiCache runs one node.

Recommendation: Add an Aurora reader in a second Availability Zone so failover takes about a minute, and enable Multi-AZ with automatic failover on ElastiCache.

REL12-BP04 Test resiliency using chaos engineering Status: Not met. Risk: Low. Effort: M. Owner: Platform lead. Backlog: HG-48. Evidence: EV-02.

The team has never injected a failure on purpose.

Recommendation: Once alarms and Multi-AZ are in place, run an AWS Fault Injection Service experiment in staging that stops the Aurora writer, with a written hypothesis and stop conditions.

REL13-BP01 Define recovery objectives for downtime and data loss Status: Not met. Risk: High. Effort: S. Owner: Head of engineering. Backlog: HG-04. Evidence: EV-01.

The business has no agreed recovery time or recovery point objective for checkout.

Recommendation: Agree an RTO and RPO for checkout and order history with the business owner, and record them in the workload's operations document.

REL13-BP02 Use defined recovery strategies to meet the recovery objectives Status: Not met. Risk: Medium. Effort: L. Owner: Platform lead. Backlog: HG-36. Evidence: EV-10.

The workload has no disaster recovery strategy beyond backups in the same Region.

Recommendation: Choose a strategy that meets the agreed objectives (backup and restore with cross-Region copies is likely enough), then document and test it.

4.4 Performance efficiency

Question	Text	Risk
PERF01	How do you select appropriate cloud resources and architecture for your workload?	None
PERF02	How do you select and use compute resources in your workload?	None
PERF05	How do your organizational practices and culture contribute to performance efficiency in your workload?	Medium

PERF01-BP06 Use benchmarking to drive architectural decisions Status: Not met. Risk: Low. Effort: M. Owner: Backend engineers. Backlog: HG-49. Evidence: EV-02.

The team chose instance and task sizes at launch and never compared them with alternatives.

Recommendation: Benchmark orders-api on Graviton-based Fargate tasks against the current x86 tasks with the load test from PERF05-BP04, and keep the faster or cheaper option.

PERF02-BP01 Select the best compute options for your workload Status: Met. Evidence: EV-13.

Containers on Fargate and an event-driven Lambda worker fit the workload's traffic shape.

PERF02-BP03 Collect compute-related metrics Status: Partial. Risk: Low. Effort: S. Owner: Platform lead. Backlog: HG-38. Evidence: EV-09.

ECS Container Insights is off, so per-task memory is invisible; the team found the INC-101 memory exhaustion in logs.

Recommendation: Enable Container Insights on the production cluster and add memory utilization to the dashboard.

PERF05-BP04 Load test your workload Status: Not met. Risk: Medium. Effort: M. Owner: Backend engineers. Backlog: HG-28. Evidence: EV-02, EV-07.

Nobody load tests before campaigns; INC-103 happened during a promotional email campaign that marketing scheduled without telling engineering.

Recommendation: Build a repeatable load test of the checkout path in staging and run it before every planned campaign; add a campaign calendar that engineering can see.

4.5 Cost optimization

Question	Text	Risk
COST02	How do you govern usage?	None
COST03	How do you monitor your cost and usage?	Medium
COST04	How do you decommission resources?	None
COST06	How do you meet cost targets when you select resource type, size and number?	Medium
COST07	How do you use pricing models to reduce cost?	None

COST02-BP05 Implement cost controls Status: Partial. Risk: Low. Effort: S. Owner: Head of engineering. Backlog: HG-39. Evidence: EV-11.

One account-level AWS Budgets alert exists and emails a former employee's address.

Recommendation: Create a budget per environment with alerts at 80 and 100 percent of forecast, sent to a team distribution list.

COST03-BP02 Add organization information to cost and usage Status: Partial. Risk: Medium. Effort: S. Owner: Platform lead. Backlog: HG-19. Evidence: EV-08, EV-11.

Terraform-managed resources carry `service` and `environment` tags, but the tags are not activated as cost allocation tags and console-created resources have none.

Recommendation: Activate the tags for cost allocation, add default_tags to the Terraform AWS provider, and add a tag policy in AWS Organizations.

COST04-BP03 Decommission resources Status: Not met. Risk: Low. Effort: S. Owner: Platform lead. Backlog: HG-40. Evidence: EV-05, EV-11.

The legacy-image-sync user and its Lambda function have been idle for months, and three unattached Elastic IP addresses and 14 old EBS snapshots remain in staging.

Recommendation: Remove the idle resources after confirming with their owners, and review unused resources monthly.

COST06-BP03 Select resource type, size, and number automatically based on metrics Status: Partial. Risk: Medium. Effort: M. Owner: Platform lead. Backlog: HG-29. Evidence: EV-11, EV-13.

storefront-web runs 2 vCPU tasks that average 9 percent CPU, and the staging environment runs at production size around the clock.

Recommendation: Right-size tasks from Container Insights data, scale staging to zero outside working hours, and review AWS Compute Optimizer recommendations each month.

COST07-BP01 Perform pricing model analysis Status: Not met. Risk: Low. Effort: S. Owner: Head of engineering. Backlog: HG-41. Evidence: EV-11.

All compute and the database run on On-Demand pricing.

Recommendation: After right-sizing, buy a Compute Savings Plan for the steady baseline and consider Aurora reserved instances for the writer.

4.6 Sustainability

Question	Text	Risk
SUS02	How do you align cloud resources to your demand?	None
SUS04	How do you take advantage of data management policies and patterns to support your sustainability goals?	None
SUS05	How do you select and use cloud hardware and services in your architecture to support your sustainability goals?	None

SUS02-BP01 Scale workload infrastructure dynamically Status: Partial. Risk: Low. Effort: S. Owner: Platform lead. Backlog: HG-42. Evidence: EV-13.

Production scales one service; staging never scales down.

Recommendation: Apply the scaling and staging schedule from REL07-BP01 and COST06-BP03.

SUS04-BP03 Use policies to manage the lifecycle of your datasets Status: Not met. Risk: Low. Effort: S. Owner: Platform lead. Backlog: HG-43. Evidence: EV-09, EV-10.

The image bucket keeps every object version forever and most CloudWatch log groups never expire.

Recommendation: Add S3 lifecycle rules for noncurrent versions and set retention on every log group (for example 30 days for application logs and longer where investigations need it).

SUS05-BP03 Use managed services Status: Met. Evidence: EV-13.

The workload runs on Fargate, Aurora, SQS and Lambda rather than self-managed servers.

4.7 DevOps lens

Question	Text	Risk
DL.SCM	Software component management	None
DL.CI	Continuous integration	None
DL.CD	Continuous delivery	Medium
DL.ADS	Advanced deployment strategies	High
DL.EAC	Everything as code	Medium
QA.FT	Functional testing	Medium
QA.ST	Security testing	Medium
O.SI	Strategic instrumentation	Medium
O.CM	Continuous monitoring	Medium
AG.SAD	Secure access and delegation	Medium
AG.CA	Continuous auditing	None
OA.STD	Supportive team dynamics	Medium
OA.BCL	Balanced cognitive load	None

DL.SCM.2 Keep feature branches short-lived Status: Partial. Risk: Low. Effort: S. Owner: Backend engineers. Backlog: HG-46. Evidence: EV-04, EV-06.

Most pull requests merge within two days, but release branches for storefront-web live for about a week while a batch of changes waits for a deployment slot.

Recommendation: Merge to main behind feature flags and deploy from main, retiring the release branches.

DL.CI.1 Integrate code changes regularly and frequently Status: Met. Evidence: EV-04.

Engineers merge small pull requests to main more than twice a week.

DL.CI.2 Trigger builds automatically upon source code modifications Status: Met. Evidence: EV-04.

Every push to main builds and pushes an image to Amazon ECR.

DL.CD.4 Automate the entire deployment process Status: Partial. Risk: Medium. Effort: M. Owner: Platform lead. Backlog: HG-31. Evidence: EV-04, EV-06.

The team ships 3.7 deployments a week, each started by hand, with the database migration run from a laptop.

Recommendation: Once the promotion pipeline from OPS05-BP10 exists, trigger production deployments from the approved staging build without any manual command.

DL.CD.6 Refine delivery pipelines using metrics for continuous improvement Status: Not met. Risk: Low. Effort: S. Owner: Head of engineering. Backlog: HG-45. Evidence: EV-06, EV-07.

Nobody tracks delivery metrics. From the exports, the change failure rate is 10.6 percent and the median time to restore service is 70 minutes.

Recommendation: Publish deployment frequency, lead time, change failure rate and time to restore from the pipeline and the incident log each month, and review the trend in the operations review.

DL.ADS.2 Implement automatic rollbacks for failed deployments Status: Not met. Risk: High. Effort: M. Owner: Platform lead. Backlog: HG-09. Evidence: EV-06, EV-13.

All 3 rollbacks in the window were manual redeployments of the previous image, started after customers or staff reported the problem.

Recommendation: Use the circuit breaker and alarm-based rollback from OPS06-BP04 for every service, including the Lambda fulfilment worker through CodeDeploy traffic shifting with alarms.

DL.ADS.3 Use staggered deployment and release strategies Status: Not met. Risk: Medium. Effort: M. Owner: Platform lead. Backlog: HG-30. Evidence: EV-04.

Every release reaches all users at once; there are no canary tasks or feature flags.

Recommendation: After blue/green is in place, shift traffic in steps (for example 10 percent for ten minutes) and release risky features behind flags.

DL.EAC.1 Organize infrastructure as code for scale Status: Partial. Risk: Medium. Effort: M. Owner: Platform lead. Backlog: HG-32. Evidence: EV-08.

Terraform covers the network, ECS and Aurora, but the team built ElastiCache, the alarms and the Lambda worker in the console, and one state file holds production and staging.

Recommendation: Import the console-built resources, split state per environment and per stack, and run plan on every pull request with drift detection each night.

QA.FT.1 Ensure individual component functionality with unit tests Status: Partial. Risk: Medium. Effort: M. Owner: Backend engineers. Backlog: HG-35. Evidence: EV-04.

orders-api has unit tests with about 40 percent line coverage; the other two services have none.

Recommendation: Cover checkout, payment status handling and the fulfilment message format first, and fail the build when the tests fail.

QA.ST.4 Enhance source code security with static application security testing Status: Not met. Risk: Medium. Effort: S. Owner: Backend engineers. Backlog: HG-22. Evidence: EV-04.

No static application security testing runs on any repository.

Recommendation: Delivered by SEC11-BP02; track it here so the DevOps lens score reflects it.

QA.ST.6 Validate third-party components using software composition analysis Status: Not met. Risk: Medium. Effort: S. Owner: Backend engineers. Backlog: HG-23. Evidence: EV-04, EV-12.

Dependencies are never scanned before release; Inspector only finds image vulnerabilities after the image is in Amazon ECR.

Recommendation: Add dependency scanning to pull requests and enable Dependabot security updates on all repositories.

O.SI.3 Instrument all systems for comprehensive telemetry data collection Status: Partial. Risk: Medium. Effort: M. Owner: Backend engineers. Backlog: HG-33. Evidence: EV-09.

Services write unstructured logs and no traces, so nobody can follow a slow checkout across storefront-web, orders-api and the database.

Recommendation: Adopt structured JSON logs with a request ID and add OpenTelemetry tracing through the AWS Distro for OpenTelemetry collector to AWS X-Ray.

O.CM.3 Conduct post-incident analysis for continuous improvement Status: Partial. Risk: Medium. Effort: S. Owner: Head of engineering. Backlog: HG-21. Evidence: EV-07.

2 of 7 incidents have an analysis, and nobody tracked any action from them to completion.

Recommendation: Delivered by OPS11-BP02; review open actions in the weekly operations review.

AG.SAD.3 Treat pipelines as production resources Status: Not met. Risk: Medium. Effort: S. Owner: Platform lead. Backlog: HG-20. Evidence: EV-04, EV-05.

Any repository admin can edit the deploy workflows, and the workflows run with the ci-deployer key that holds AdministratorAccess in production.

Recommendation: Protect the workflow files with CODEOWNERS and required reviews, and run production deployments only from a GitHub environment with required reviewers.

AG.CA.1 Establish comprehensive audit trails Status: Partial. Risk: Low. Effort: S. Owner: Platform lead. Backlog: HG-44. Evidence: EV-06, EV-09.

The organization trail records API calls, but nothing links a production change to its pull request, approver and pipeline run.

Recommendation: Record the commit SHA, pull request and approver on every deployment (for example as ECS task definition tags and a GitHub deployment record), and keep the records for at least a year.

OA.STD.6 Provide teams ownership of the entire value stream for their product Status: Partial. Risk: Medium. Effort: M. Owner: Head of engineering. Backlog: HG-34. Evidence: EV-02.

Backend engineers own the code, but the platform lead starts every production deployment and runs every database migration, so releases wait for one person.

Recommendation: Once deployments run from the pipeline, let each service's engineers approve and release their own changes, with the platform lead reviewing only infrastructure changes.

OA.BCL.7 Cultivate a psychologically-safe culture for experimentation Status: Partial. Risk: Low. Effort: S. Owner: Head of engineering. Backlog: HG-47. Evidence: EV-02, EV-07.

Engineers described incident reviews that start from who made the change, and said they batch releases to avoid being the one on the call.

Recommendation: Run post-incident analyses in a blameless format that asks what made the failure possible, and have the head of engineering open each review by stating that rule.

5. Prioritized backlog

The scripts rank items by risk, then by effort so quick wins lead each risk band. Effort is S (up to two days), M (up to two weeks) or L (more than two weeks) for one engineer. The full backlog, with dependencies and recommendations, is in `evidence/backlog.csv` for import into a tracker, and in `evidence/backlog.md`.

Rank	Key	Best practice	Title	Risk	Effort	Owner	Days
1	HG-01	OPS06-BP01	Plan for unsuccessful changes	High	S	Platform lead	30
2	HG-02	SEC02-BP02	Use temporary credentials	High	S	Platform lead	30
3	HG-03	REL09-BP04	Perform periodic recovery of the data to verify backup integrity and processes	High	S	Platform lead	30
4	HG-04	REL13-BP01	Define recovery objectives for downtime and data loss	High	S	Head of engineering	30
5	HG-05	OPS06-BP04	Automate testing and rollback	High	M	Platform lead	30
6	HG-06	SEC03-BP02	Grant only the access each role needs	High	M	Platform lead	30
7	HG-07	REL06-BP01	Monitor all components for the workload (Generation)	High	M	Platform lead	30

Rank	Key	Best practice	Title	Risk	Effort	Owner	Days
8	HG-08	REL10-BP01	Deploy the workload to multiple locations	High	M	Platform lead	30
9	HG-09	DL.ADS.2	Implement automatic rollbacks for failed deployments	High	M	Platform lead	30
10	HG-10	OPS04-BP01	Identify key performance indicators	Medium	S	Head of engineering	30
11	HG-11	OPS07-BP03	Use runbooks to perform procedures	Medium	S	Platform lead	60
12	HG-12	OPS10-BP01	Use a process for event, incident, and problem management	Medium	S	Head of engineering	60
13	HG-13	OPS11-BP02	Perform post-incident analysis	Medium	S	Head of engineering	60
14	HG-14	SEC02-BP03	Store and use secrets securely	Medium	S	Backend engineers	60
15	HG-15	SEC04-BP01	Configure service and application logging	Medium	S	Platform lead	60
16	HG-16	SEC06-BP01	Perform vulnerability management	Medium	S	Platform lead	60
17	HG-17	SEC10-BP02	Develop incident management plans	Medium	S	Head of engineering	60
18	HG-18	REL07-BP01	Use automation when obtaining or scaling resources	Medium	S	Platform lead	60

Rank	Key	Best practice	Title	Risk	Effort	Owner	Days
19	HG-19	COST03-BP02	Add organization information to cost and usage	Medium	S	Platform lead	60
20	HG-20	AG.SAD.3	Treat pipelines as production resources	Medium	S	Platform lead	60
21	HG-21	O.CM.3	Conduct post-incident analysis for continuous improvement	Medium	S	Head of engineering	60
22	HG-22	QA.ST.4	Enhance source code security with static application security testing	Medium	S	Backend engineers	60
23	HG-23	QA.ST.6	Validate third-party components using software composition analysis	Medium	S	Backend engineers	60
24	HG-24	OPS05-BP02	Test and validate changes	Medium	M	Backend engineers	60
25	HG-25	OPS05-BP10	Fully automate integration and deployment	Medium	M	Platform lead	60
26	HG-26	OPS06-BP03	Employ safe deployment strategies	Medium	M	Platform lead	60
27	HG-27	SEC11-BP02	Automate testing throughout the development and release lifecycle	Medium	M	Backend engineers	60
28	HG-28	PERF05-BP04	Load test your workload	Medium	M	Backend engineers	60

Rank	Key	Best practice	Title	Risk	Effort	Owner	Days
29	HG-29	COST06-BP03	Select resource type, size, and number automatically based on metrics	Medium	M	Platform lead	60
30	HG-30	DL.ADS.3	Use staggered deployment and release strategies	Medium	M	Platform lead	60
31	HG-31	DL.CD.4	Automate the entire deployment process	Medium	M	Platform lead	60
32	HG-32	DL.EAC.1	Organize infrastructure as code for scale	Medium	M	Platform lead	60
33	HG-33	O.SI.3	Instrument all systems for comprehensive telemetry data collection	Medium	M	Backend engineers	60
34	HG-34	OA.STD.6	Provide teams ownership of the entire value stream for their product	Medium	M	Head of engineering	60
35	HG-35	QA.FT.1	Ensure individual component functionality with unit tests	Medium	M	Backend engineers	60
36	HG-36	REL13-BP02	Use defined recovery strategies to meet the recovery objectives	Medium	L	Platform lead	90
37	HG-37	OPS08-BP01	Analyze workload metrics	Low	S	Head of engineering	90

Rank	Key	Best practice	Title	Risk	Effort	Owner	Days
38	HG-38	PERF02-BP03	Collect compute-related metrics	Low	S	Platform lead	60
39	HG-39	COST02-BP05	Implement cost controls	Low	S	Head of engineering	90
40	HG-40	COST04-BP03	Decommission resources	Low	S	Platform lead	90
41	HG-41	COST07-BP01	Perform pricing model analysis	Low	S	Head of engineering	90
42	HG-42	SUS02-BP01	Scale workload infrastructure dynamically	Low	S	Platform lead	90
43	HG-43	SUS04-BP03	Use policies to manage the lifecycle of your datasets	Low	S	Platform lead	90
44	HG-44	AG.CA.1	Establish comprehensive audit trails	Low	S	Platform lead	90
45	HG-45	DL.CD.6	Refine delivery pipelines using metrics for continuous improvement	Low	S	Head of engineering	90
46	HG-46	DL.SCM.2	Keep feature branches short-lived	Low	S	Backend engineers	90
47	HG-47	OA.BCL.7	Cultivate a psychologically-safe culture for experimentation	Low	S	Head of engineering	90
48	HG-48	REL12-BP04	Test resiliency using chaos engineering	Low	M	Platform lead	90

Rank	Key	Best practice	Title	Risk	Effort	Owner	Days
49	HG-49	PERF01-BP06	Use benchmarking to drive architectural decisions	Low	M	Backend engineers	90

6. 30/60/90-day roadmap

High risks with small or medium effort start in the first 30 days. High risks with large effort, and medium risks with small or medium effort, follow in days 31 to 60. Everything else lands in days 61 to 90. An item that other work depends on moves forward into the bucket of the earliest item that needs it, and within a bucket every item follows the items it depends on.

First 30 days (10 items)

- HG-01 Plan for unsuccessful changes (OPS06-BP01, High, S), Platform lead.
- HG-02 Use temporary credentials (SEC02-BP02, High, S), Platform lead.
- HG-03 Perform periodic recovery of the data to verify backup integrity and processes (REL09-BP04, High, S), Platform lead.
- HG-04 Define recovery objectives for downtime and data loss (REL13-BP01, High, S), Head of engineering.
- HG-06 Grant only the access each role needs (SEC03-BP02, High, M), Platform lead. After HG-02.
- HG-07 Monitor all components for the workload (Generation) (REL06-BP01, High, M), Platform lead.
- HG-08 Deploy the workload to multiple locations (REL10-BP01, High, M), Platform lead.
- HG-10 Identify key performance indicators (OPS04-BP01, Medium, S), Head of engineering. Moved forward because later work depends on it.
- HG-05 Automate testing and rollback (OPS06-BP04, High, M), Platform lead. After HG-10.
- HG-09 Implement automatic rollbacks for failed deployments (DL.ADS.2, High, M), Platform lead. After HG-05.

Days 31 to 60 (26 items)

- HG-11 Use runbooks to perform procedures (OPS07-BP03, Medium, S), Platform lead.
- HG-12 Use a process for event, incident, and problem management (OPS10-BP01, Medium, S), Head of engineering.
- HG-13 Perform post-incident analysis (OPS11-BP02, Medium, S), Head of engineering. After HG-12.
- HG-14 Store and use secrets securely (SEC02-BP03, Medium, S), Backend engineers.
- HG-15 Configure service and application logging (SEC04-BP01, Medium, S), Platform lead.
- HG-16 Perform vulnerability management (SEC06-BP01, Medium, S), Platform lead.
- HG-17 Develop incident management plans (SEC10-BP02, Medium, S), Head of engineering. After HG-12.
- HG-18 Use automation when obtaining or scaling resources (REL07-BP01, Medium, S), Platform lead.
- HG-19 Add organization information to cost and usage (COST03-BP02, Medium, S), Platform lead.
- HG-20 Treat pipelines as production resources (AG.SAD.3, Medium, S), Platform lead. After HG-06.
- HG-21 Conduct post-incident analysis for continuous improvement (O.CM.3, Medium, S), Head of engineering. After HG-13.
- HG-23 Validate third-party components using software composition analysis (QA.ST.6, Medium, S), Backend engineers.
- HG-24 Test and validate changes (OPS05-BP02, Medium, M), Backend engineers.
- HG-25 Fully automate integration and deployment (OPS05-BP10, Medium, M), Platform lead. After HG-24.
- HG-26 Employ safe deployment strategies (OPS06-BP03, Medium, M), Platform lead. After HG-05.
- HG-27 Automate testing throughout the development and release lifecycle (SEC11-BP02, Medium, M), Backend engineers. After HG-24.

- HG-22 Enhance source code security with static application security testing (QA.ST.4, Medium, S), Backend engineers. After HG-27.
- HG-28 Load test your workload (PERF05-BP04, Medium, M), Backend engineers.
- HG-30 Use staggered deployment and release strategies (DL.ADS.3, Medium, M), Platform lead. After HG-26.
- HG-31 Automate the entire deployment process (DL.CD.4, Medium, M), Platform lead. After HG-25.
- HG-32 Organize infrastructure as code for scale (DL.EAC.1, Medium, M), Platform lead.
- HG-33 Instrument all systems for comprehensive telemetry data collection (O.SI.3, Medium, M), Backend engineers. After HG-07.
- HG-34 Provide teams ownership of the entire value stream for their product (OA.STD.6, Medium, M), Head of engineering. After HG-31.
- HG-35 Ensure individual component functionality with unit tests (QA.FT.1, Medium, M), Backend engineers. After HG-24.
- HG-38 Collect compute-related metrics (PERF02-BP03, Low, S), Platform lead. Moved forward because later work depends on it.
- HG-29 Select resource type, size, and number automatically based on metrics (COST06-BP03, Medium, M), Platform lead. After HG-38.

Days 61 to 90 (13 items)

- HG-36 Use defined recovery strategies to meet the recovery objectives (REL13-BP02, Medium, L), Platform lead. After HG-04, HG-03.
- HG-37 Analyze workload metrics (OPS08-BP01, Low, S), Head of engineering. After HG-10.
- HG-39 Implement cost controls (COST02-BP05, Low, S), Head of engineering.
- HG-40 Decommission resources (COST04-BP03, Low, S), Platform lead.
- HG-41 Perform pricing model analysis (COST07-BP01, Low, S), Head of engineering. After HG-29.
- HG-42 Scale workload infrastructure dynamically (SUS02-BP01, Low, S), Platform lead. After HG-18.
- HG-43 Use policies to manage the lifecycle of your datasets (SUS04-BP03, Low, S), Platform lead.
- HG-44 Establish comprehensive audit trails (AG.CA.1, Low, S), Platform lead. After HG-15.
- HG-45 Refine delivery pipelines using metrics for continuous improvement (DL.CD.6, Low, S), Head of engineering.
- HG-46 Keep feature branches short-lived (DL.SCM.2, Low, S), Backend engineers.
- HG-47 Cultivate a psychologically-safe culture for experimentation (OA.BCL.7, Low, S), Head of engineering. After HG-13.
- HG-48 Test resiliency using chaos engineering (REL12-BP04, Low, M), Platform lead. After HG-07, HG-08.
- HG-49 Use benchmarking to drive architectural decisions (PERF01-BP06, Low, M), Backend engineers. After HG-28.

7. Evidence register

ID	Kind	Description	Source file	Cited by
EV-01	interview	Scoping workshop with the head of engineering and the platform lead	-	3
EV-02	interview	Two-hour architecture and operations interview with the platform lead and two backend engineers	-	9

ID	Kind	Description	Source file	Cited by
EV-03	interview	Support interview with the customer support lead about outages and how customers report them	-	2
EV-04	review	Read-only review of the GitHub Actions workflows in the storefront and orders repositories	-	17
EV-05	export	IAM credential report for the production account (synthetic extract)	data/synthetic/exports/credential-report.csv	4
EV-06	export	Deployment history for the observation window (synthetic extract)	data/synthetic/exports/deployments.csv	8
EV-07	export	Incident log for the observation window (synthetic extract)	data/synthetic/exports/incidents.csv	9
EV-08	review	Read-only review of the Terraform repository, its state layout and drift against the account	-	5
EV-09	review	Read-only review of CloudWatch dashboards, alarms and log groups	-	8
EV-10	review	Read-only review of the Aurora cluster, AWS Backup plans and restore history	-	6
EV-11	review	Read-only review of Cost Explorer by service and by tag, and of AWS Budgets	-	5
EV-12	review	Read-only review of AWS Security Hub and Amazon Inspector findings	-	4
EV-13	review	Read-only review of the ECS services, task sizes and scaling policies	-	9

8. Limits and next steps

- The findings describe the workload during a 90-day observation window. A re-review after the first 90 days should confirm which high risks the team has closed.

- The review read configuration and history; it did not test failover, run load tests or scan for vulnerabilities itself. Those are backlog items.
- In an engagement, the answers are also recorded in the AWS Well-Architected Tool in the client's own account, and this report links to that workload. `make test-live` shows the same answers round-tripping through the Tool.

Fictional sample prepared to show the format of an AWS Well-Architected and DevOps assessment. Harbor Goods, its people, accounts and incidents are fictional. Generated from `data/synthetic/` by the scripts in `scripts/`.